Лекція №24 (2 години). Огляд TypeScript: типізація у веб-розробці.

План лекції

1. Проблема динамічної типізації в JavaScript (що таке Runtime Exceptions).
2. Що таке TypeScript: концепція надмножини (Superset) та компіляція.
3. Базові типи (String, Number, Boolean, Array).
4. Типізація об'єктів: Інтерфейси (Interfaces) та Типи (Type Aliases).
5. Типізація функцій (аргументи та значення, що повертається) та стрілочні функції.
6. Опціональні параметри (Optional parameters) та Union типи (`|`).
7. Базове використання TypeScript в інтеграції з React (типізація Props).
8. Переваги TypeScript: розробка з підказками IDE (Intellisense) та раннє виявлення помилок.

Перелік умовних скорочень

- **JS** (JavaScript) — мова програмування з динамічною типізацією.
- **TS** (TypeScript) — типізоване надмножина (superset) JavaScript (мова розроблена Microsoft).
- **IDE** (Integrated Development Environment) — інтегроване середовище розробки (напр., VS Code).
- **DOM** (Document Object Model) — об'єктна модель документа.
- **API** (Application Programming Interface) — інтерфейс прикладного програмування.

Вступ

Протягом усього курсу ми писали код "чистим" (ванільним) **JavaScript**. Одна з його найголовніших особливостей полягає в тому, що він є мовою з **динамічною типізацією**. Це означає, що коли ви створюєте змінну `let x = 10`, JavaScript мовчки дозволить вам через дві строчки написати `x = "Привіт"`, а потім `x = []`. JavaScript байдуже, що лежить у змінній.

Для маленьких скриптів це дуже зручно і швидко. Але коли ваш проект виростає до тисяч файлів і десятків розробників у команді, ця лояльність обертається справжнім жахом. Уявіть, що Марія (Backend Developer) написала функцію для розрахунку знижки `calculateDiscount(price)`. А Іван (Frontend Developer) випадково передав у неї рядок `calculateDiscount("100 доларів")`. JavaScript не скаже Івану про помилку під час написання коду. Додаток здаватиметься робочим, Іван відправить його на сервер, і вже там, ОНЛАЙН, перед справжнім клієнтом, браузер спробує поділити

рядок на число і видасть `NaN` (або `TypeError: undefined is not a function`). Додаток епічно "впаде" (Runtime Exception).

Як цьому запобігти? Нам потрібен суворий контрольор, який перевірить типи всіх змінних ЩЕ ДО того, як ви запустите код. Знайомтесь — **TypeScript**.

Мета цієї лекції — зрозуміти філософію суворої типізації та розібрати синтаксис інструменту, який на сьогодні став обов'язковим "законним" стандартом для всієї світової Frontend та Backend розробки.

1. Що таке TypeScript?

TypeScript (TS) — це мова програмування (а точніше — лінтер/аналізатор на стероїдах), розроблена компанією **Microsoft** (автором мови C#).

Головне правило TypeScript: **Будь-який правильний JavaScript-код є правильним TypeScript-кодом**. Звучить дивно? Справа в тому, що TS — це просто **Надмножина (Superset)**. Він бере звичайний JS і просто "наклеює" поверх нього свої нові можливості (анотації типів).

Як це працює фізично:

1. Ви створюєте файл з розширенням `.ts` (або `.tsx` для React) замість `.js`.
2. Ви пишете код з використанням суворих типів.
3. До того, як код потрапить у браузер, втручається програма-компілятор (`tsc` або Vite). Вона сканує ваш код на наявність логічних помилок (де ви намагаєтесь поділити рядок на масив).
4. Якщо є помилка — компіляція зупиняється. Код врятовано від релізу!
5. Якщо все ідеально, компілятор просто "вирізає" ножицями (стирає) всі згадки про типи з вашого коду, і на виході створюється 100% звичайний файл `bundle.js`, який відправляється у браузер.

Браузер НІКОЛИ не бачить і не знає про існування TypeScript. Він виконує тільки чистий згенерований JavaScript.

2. Базові типи (Primitives & Arrays)

Щоб повідомити компілятору, який саме тип даних ви очікуєте, ставиться двокрапка `:` одразу після назви змінної.

```
// 1. Примітиви (String, Number, Boolean)
let userName: string = "Петро";
let userAge: number = 25;
let isActive: boolean = true;

// Якщо ви спробуєте порушити правило, VS Code одразу підкреслить код червоним:
```

```
// userName = 500;
// ^^^ Помилка: Тип 'number' неможливо призначити типу 'string'

// 2. Масиви
// Треба вказати, з ЧОГО складається масив. Додаємо [] після типу:
let prices: number[] = [100, 200, 300];
let names: string[] = ["Київ", "Львів", "Одеса"];

// prices.push("Безкоштовно");
// ^^^ Помилка
```

Примітка: У 90% випадків TS настільки розумний (Type Inference - Виведення типів), що якщо ви одразу ініціалізуєте змінну, йому не треба нічого писати. Код `let city = "Київ";` *автоматично буде вважатися* `string` *назавжди.*

3. Типізація об'єктів (Interfaces та Type Aliases)

Найголовніша сила TS розкривається під час роботи з об'єктами. Коли ви отримуєте об'єкт користувача з сервера, звідки вам знати, як правильно написати в коді: `user.firstName` чи `user.first_name`? Доводиться відкривати Postman і гуглити структуру бази.

З TS ми ПРОЕКТУЄМО форму об'єктів заздалегідь, створюючи "Контракти" за допомогою інструменту **Interface** або **Type**. Вони майже ідентичні.

```
// Створюємо креслення (Інтерфейс) для Юзера
interface User {
  id: number;
  firstName: string;
  lastName: string;
  age: number;
  isAdmin: boolean;
}

// Тепер ми створюємо реальний об'єкт і зобов'язуємо його відповідати кресленню
User
const myCurrentBoss: User = {
  id: 1,
  firstName: "Ілон",
  lastName: "Маск",
  age: 53,
  isAdmin: true,
```

```
};

// Контроль працює в обидві сторони!
// 1. Якщо ви забудете написати 'isAdmin', об'єкт засвітиться червоним ("Відсутня властивість isAdmin").
// 2. Якщо ви напишете console.log(myCurrentBoss.emaillll), код не скомпілюється ("Властивості 'emaillll' не існує в типі 'User'").
```

Ще одна неймовірна перевага: коли ви поставите крапку `myCurrentBoss.`, редактор VS Code (який збудований на TS) миттєво відкриє вам список з усіма доступними полями (`Intellisense`), бо він знає структуру! Вам не треба нічого запам'ятовувати.

4. Опціональні параметри (Optional) та Union типи (|)

Іноді в об'єкті деякі поля не є обов'язковими. Наприклад, в Інтернет-магазині юзер може вказати свій номер телефону, а може й ні. Щоб TS не сварився на відсутність поля, ставимо знак питання `?` перед двокрапкою.

```
interface Customer {
  id: number;
  name: string;
  phone?: string; // Це поле не є обов'язковим (може бути string або undefined)
}
```

А що робити, якщо якась змінна може бути двох абсолютно різних типів? Наприклад, ID з бази може прийти як число `15`, а може прийти як рядок у форматі UUID `'abc-45-de'`. Ми використовуємо **Union Types** (Об'єднання типів) через вертикальну лінію `|`.

```
// Змінна може бути АБО числом, АБО рядком
let serverResponseId: number | string;

serverResponseId = 404; // OK
serverResponseId = "Success"; // OK
// serverResponseId = true;   // ПОМИЛКА
```

Дуже часто Union типи використовують для жорсткого обмеження конкретних значень-рядків (Літеральні типи):

```
let userStatus: "online" | "offline" | "banned";
userStatus = "online"; // OK
```

```
// userStatus = "playing"; // ПОМИЛКА: Тип '"playing"' не належить до трьох дозволених.
```

5. Типізація функцій

Під час написання функцій ми повинні типізувати ДВІ речі:

1. Що функція ДО СЕБЕ приймає (аргументи).
2. Що функція ВІД СЕБЕ віддає через `return` (значення повернення).

```
// 1. Аргументи (message: string, count: number)
// 2. Значення повернення (: string)
function buildMessage(message: string, count: number): string {
  return `${message}. Повторень: ${count}`;
}

const finalGreeting = buildMessage("Привіт", 5);

// Типізація стрілочної функції
const multiply = (x: number, y: number): number => {
  return x * y;
};
```

Якщо ваша функція нічого не повертає назовні `return` (просто робить `console.log` або змінює глобальну змінну), її тип повернення називається **`void`** (порожнеча).

```
function logError(errorMsg: string): void {
  console.error(`Увага: ${errorMsg}`);
}
```

6. TypeScript у React (Типізація Props)

Найбільший біль у великому React-додатку — це намагатися з'ясувати, які саме Пропси чекає компонент `<ProductCard>`, який написав ваш колега півроку тому.

З TypeScript ми створюємо `interface` спеціально для Пропсів цього компонента.

```
// файл ProductCard.tsx (зверніть увагу на розширення .tsx!)
import React from "react";

// 1. Описуємо "Очікування" (Контракт)
interface ProductCardProps {
  title: string;
  price: number;
  imageUrl?: string; // Картинка необов'язкова
  isDiscounted: boolean;
}

// 2. Застосовуємо ці очікування до аргументу props (через деструктуризацію)
export function ProductCard({
  title,
  price,
  imageUrl,
  isDiscounted,
}: ProductCardProps) {
  return (
    <div className="card">
      {imageUrl ? (
        <img src={imageUrl} alt={title} />
      ) : (
        <div className="placeholder" />
      )}
      <h3>{title}</h3>
      <p>
        Ціна: {price} грн {isDiscounted && <span>(Зі знижкою!)</span>}
      </p>
    </div>
  );
}
```

Що тепер станеться в `App.tsx`? Як тільки ви напишете `<ProductCard>`, VS Code миттєво підкреслить вам його червоним з повідомленням: "*Гей, ти забув передати обов'язковий пропс title!*". Коли ви почнете вводити `title="..."`, редактор автоматично підкаже, які ще пропси можна передати і їхній тип (`price` вимагатиме число, а не рядок).

Висновки

1. **Динамічна типізація JavaScript** дозволяє змінним з часом отримувати будь-які типи даних (число може стати об'єктом). На великих проектах це невідворотно призводить до катастрофічних логічних помилок (Runtime Exceptions) у браузері користувача через невірні дані.
2. **TypeScript (TS)** — це розроблений Microsoft компілятор та інструмент суворого статичного аналізу коду, який надбудовує систему типів поверх звичайного JS (Superset).
3. TS перевіряє код на етапі збереження у редакторі або зборки проекту. Фінальний скомпільований код, який читає браузер — це завжди 100% чистий (ванільний) JavaScript, у якому немає жодного слова з TS-коду. Усі типи "вирізаються".
4. Головна зброя розробника — **Інтерфейси (`interface`)** або Типи. Вони створюють суворі інженерні креслення об'єктів (наприклад, описують сутність "Користувач").
5. Маркер питання `?` біля ключа `phone?: string;` створює Опціональне поле, яке можна передати, а можна і пропустити. Об'єднання `|` у правилі "Union Types" каже системі, що змінна може належати до різних родин типів (наприклад `string | number`).
6. Будь-яка власноруч написана функція у TypeScript вимагає типізації аргументів `(name: string)` та обов'язкового опису вихідного значення після дужок `(): void` .
7. У React екосистемі TypeScript миттєво вирішує головну проблему архітектури — невідомість Пропсів. Під час передачі невірного типу у властивості `<Button size={10} />` редактор миттєво підкреслить атрибут червоним (якщо за інструкцією розмір там має бути рядком `"large"`).

Джерела

1. TypeScript Handbook — офіційна інтерактивна книга від Microsoft.
2. React + TypeScript Cheatsheet — Золотий Стандарт всієї планети щодо того, які типи використовувати для специфічних React подій (Event Handlers).
3. Matt Pocock: Total TypeScript — найкраща навчальна платформа для опанування TS рівня Сеньйора від сучасного експерта.

Запитання для самоперевірки

1. Згадайте, що таке JavaScript помилка "Runtime Exception". В який момент часу і де саме вона "вибухає" для звичайного Юзера, коли ми просто використовуємо ванільний JS? А коли TS зупиняє цю помилку?
2. Браузер (Chrome чи Safari) має вбудований безкоштовний компілятор TypeScript, щоб читати цей новий синтаксис? Що фізично потрапить у кінцевий файл `bundle.js` , який віддає сервер на хостингу?
3. Що у філософії TypeScript означає знак питання `?` відразу після імені ключа об'єкта (в межах "Interface")? Яке значення JS може ховатися на цьому місці окрім бажаного стрінгу (рядка)?

4. Поясніть концепцію та призначення оператора вертикальної лінії `|` під назвою "Union type". Коли це може знадобитися у роботі з функцією приймання ідентифікаторів товарів з бази даних?
5. Для чого ставиться ключове слово `void` після закриваючої круглої дужки функції `function sayHello(...) : void`?
6. Якщо ваш компонент `UserCard` приймає в себе пропси імені, та статусу (active чи не active), як за допомогою TypeScript захистити іншого розробника від того, щоб він випадково не передав цей `status` у форматі звичайного стрінгу "OK"? Складіть цей "Контракт Пропсів" в голові.